

Deep Transfer Learning for Six-Class Fruit Freshness Classification: An Output-Audited Experimental Study

Abstract—Automatic recognition of fruit freshness can support quality inspection where visual screening is repetitive and subjective. This paper documents an image-classification project that separates fresh and rotten apples, bananas, and oranges into six classes. Rather than presenting every experimental branch as successful, the study audits the executable notebook and reports only results that are directly supported by recorded outputs. The main benchmark uses four frozen ImageNet backbones—VGG16, ResNet50, EfficientNetB0, and DenseNet121—with a shared six-class classification head and a common directory-generator input protocol. DenseNet121 produced the strongest result in this four-model benchmark, reaching 98.18% test accuracy, compared with 95.14% for VGG16, 38.88% for ResNet50, and 22.28% for EfficientNetB0. A separate VGG16 baseline experiment later achieved 98.63% test accuracy, but it used a different classifier head and is therefore reported separately rather than merged into the main benchmark. Image-quality checks using Gaussian smoothing and CLAHE yielded PSNR values of 23.67–26.76 dB and SSIM values of 0.754–0.938 on three recorded samples. The notebook also contains attention, feature-fusion, Grad-CAM, ShuffleNetV2 with multi-head self-attention, and five-fold cross-validation branches; however, several of these were incomplete or failed at runtime. The resulting paper provides a reproducible account of what the implementation actually demonstrates, including methodological caveats that materially affect interpretation of the results.

Index Terms—fruit freshness, image classification, transfer learning, DenseNet121, VGG16, convolutional neural network, image augmentation, output auditing

I. INTRODUCTION

Fruit freshness assessment is an important quality-control task in retail, storage, and food-supply workflows. Visual inspection remains common, yet its consistency depends on lighting, viewpoint, operator experience, and the subtle appearance changes that occur during spoilage. Recent work has therefore explored convolutional neural networks (CNNs), transfer learning, object detection, and multi-task learning for fruit and vegetable freshness recognition [1]–[10].

Transfer learning is particularly attractive when a project dataset is modest relative to large-scale vision corpora: features learned from ImageNet can be reused and adapted to a new task [11], [12]. This idea underlies widely used backbones such as VGG [13], ResNet [14], DenseNet [15], EfficientNet [16], and MobileNetV2 [17]. Lightweight networks and attention mechanisms provide additional design options when computational cost or selective feature emphasis matters [18], [19], [21].

This project investigates six-way fruit freshness classification using images of fresh and rotten apples, bananas, and oranges. Its contribution is not a new network architecture. Instead, it is an output-audited empirical study of a substantial experimental notebook. The notebook includes preprocessing checks, a transfer-learning comparison, offline augmentation, a VGG16 attention attempt, feature fusion, Grad-CAM, a ShuffleNetV2–multi-head self-attention experiment, and five-fold cross-validation code. Some branches complete successfully, while others terminate with runtime errors or interruptions. Reporting those branches as equally successful would misrepresent the implementation.

Accordingly, this paper has three goals: (1) reconstruct the actual methodology used by the notebook; (2) report the completed outputs with their exact recorded values; and (3) separate verified findings from exploratory or invalidated branches. This distinction is especially important because post-hoc evaluation code can look plausible while still producing unreliable metrics if prediction order and label order are not synchronized.

II. RELATED WORK

Palakodati *et al.* studied fresh/rotten fruit recognition with CNN and transfer learning [1]. Fahad *et al.* extended freshness categorization to fruits and vegetables using deep learning and object detection [2]. Amin *et al.* evaluated transfer learning for automatic fruit freshness classification [3], while Sia and Baco focused on CNN hyperparameter tuning for fresh and rotten fruit recognition [4]. Ahmed *et al.* also investigated deep models for fresh-versus-stale produce recognition [5]. More recent studies have considered multi-task learning [6], shelf-life-aware rotten-fruit classification [7], fusion of multiple pretrained CNNs [8], and additional CNN-based fruit classification pipelines [9], [10].

The present work differs in emphasis. It does not attempt to claim state-of-the-art accuracy. Instead, it compares the notebook’s recorded transfer-learning runs under a shared generator setup and explicitly identifies branches whose outputs cannot support quantitative claims. This kind of audit is useful because architecture comparisons are only meaningful when data handling, preprocessing, and evaluation procedures are aligned.

III. MATERIALS AND METHODS

A. Dataset and Class Definition

The notebook uses a Kaggle fruit image dataset organized into training and testing directories. Six directory labels are recorded: `freshapples`, `freshbanana`, `freshoranges`, `rottenapples`, `rottenbanana`, and `rottenoranges`. The training directory contains 10,901 images and the provided test directory contains 2,698 images. A 90/10 split inside the training directory is used by the main Keras generator protocol, yielding 9,813 training and 1,088 validation images. All main generator images are resized to 224×224 pixels and rescaled to $[0, 1]$.

Fig. 1 reproduces the sample-image panel stored in the notebook output. It illustrates the visual diversity in pose, background, surface texture, and spoilage appearance that the classifiers must handle.

B. Two Parallel Data-Handling Branches

The notebook contains two distinct data-handling paths, which should not be conflated. The first is the main directory-generator protocol used by the four transfer-learning benchmarks. The second creates an explicit stratified 80/10/10 split from the 10,901 training-directory images: 8,720 training, 1,090 validation, and 1,091 test images. The second branch then creates exactly two augmented variants per training image, producing 17,440 synthetic images and 26,160 original-plus-augmented samples.

Crucially, the resulting augmented arrays are not passed into the subsequent VGG16, ResNet50, EfficientNetB0, or DenseNet121 training cells; those models continue to consume `train_data`, `val_data`, and `test_data` from the directory generators. Therefore, augmentation is documented as a completed preprocessing experiment, not as a causal explanation for the main benchmark accuracies. Image augmentation is widely used to improve diversity and regularization in deep learning [22], but its benefit cannot be attributed to a model unless the augmented samples are actually included in the model's training stream.

C. Image-Quality Audit

A separate preprocessing function resizes images to 224×224 , applies a 3×3 Gaussian blur, converts BGR to LAB, and applies CLAHE to the *L* channel using a clip limit of 2.0 and an 8×8 tile grid. This function is used for an image-quality audit, not as the input transformation for the main generator models.

The notebook records peak signal-to-noise ratio (PSNR) and structural similarity (SSIM) between the resized original and enhanced versions of three fresh-banana samples. PSNR is based on mean squared error,

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right), \quad (1)$$

while SSIM compares local luminance, contrast, and structure [23], [24]. The recorded values are shown in Fig. 3.

D. Transfer-Learning Benchmark

Four ImageNet-pretrained CNNs are used as frozen feature extractors: VGG16, ResNet50, EfficientNetB0, and DenseNet121 [13]–[16]. Each backbone is followed by global average pooling, a 128-unit ReLU layer with L2 regularization, dropout of 0.5, and a six-unit softmax classifier. The networks are optimized with Adam [25] and categorical cross-entropy. Early stopping and checkpointing are configured around validation loss.

The main generator only applies generic $1/255$ rescaling. Model-specific preprocessing functions from the corresponding application families are not used in these benchmark cells. Consequently, the results should be interpreted as performance under the notebook's common rescaling protocol, not as an optimized or fully standardized comparison of the underlying architectures.

E. Exploratory Extensions

The notebook also contains several extensions. A separate VGG16 baseline uses a flattened backbone output, a 256-unit dense layer, dropout, and softmax. An attempted CBAM-style attention model follows the idea of channel/spatial attention [19]; however, the construction fails because raw TensorFlow operations are applied to Keras symbolic tensors. A VGG16+ResNet50 feature-fusion model is constructed and can produce batch predictions, but the notebook does not record a complete training/evaluation table for it. A Grad-CAM branch, conceptually based on [20], later fails because the active `model` variable has been reassigned to a non-Keras PyTorch object.

The PyTorch branch combines a frozen ShuffleNetV2 backbone [18] with multi-head self-attention inspired by transformer attention [21]. It uses ImageNet-style normalization, Adam optimization, weight decay, and a learning-rate scheduler. Five epochs are recorded before a keyboard interruption. Finally, code for five-fold MobileNetV2 cross-validation [17] is present but has no recorded output.

IV. RESULTS

A. Preprocessing Audit

The three image-quality samples produced PSNR values of 23.6652, 23.8792, and 26.7629 dB, with SSIM values of 0.7540, 0.8165, and 0.9377, respectively. These values show that the enhanced images retain substantial structural similarity while changing local contrast. Because this audit branch is separate from the main generator, these metrics characterize the preprocessing experiment only.

B. Main Four-Model Benchmark

Table I reports the exact train, validation, and test outputs stored by the notebook. DenseNet121 is the strongest of the four runs, with 98.18% test accuracy and 0.1931 test loss. VGG16 follows with 95.14% test accuracy. ResNet50 and EfficientNetB0 perform substantially worse under the same recorded generator setup.

The gap between architectures is unusually large. It would be tempting to conclude that DenseNet121 is inherently superior

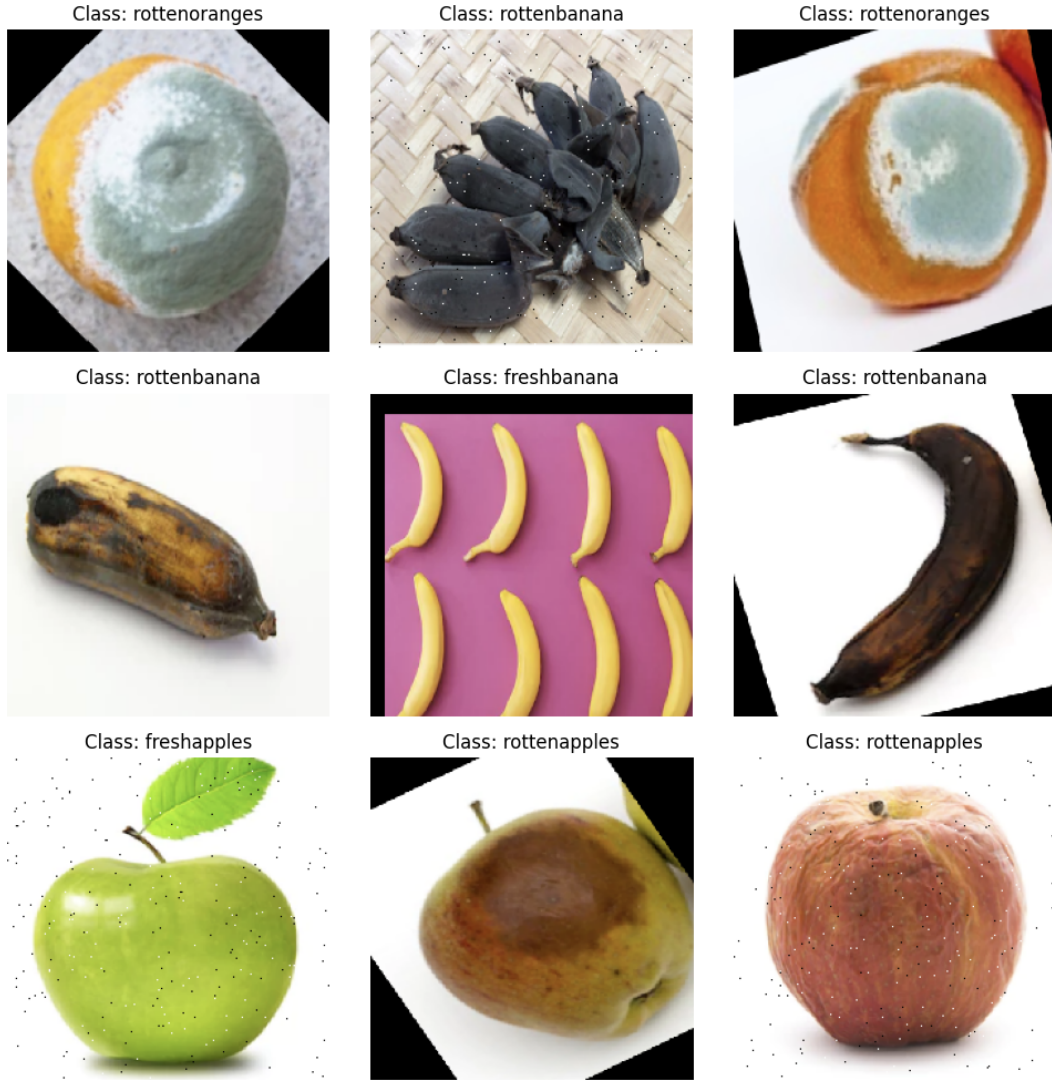


Fig. 1: Sample training images reproduced from the notebook output. The dataset contains six classes: fresh and rotten apples, bananas, and oranges.

TABLE I: Recorded outputs for the four main transfer-learning runs

Model	Total Params	Trainable Params	Train Acc. (%)	Train Loss	Val. Acc. (%)	Val. Loss	Test Acc./Loss
VGG16	14,781,126	66,438	95.73	0.2330	96.60	0.2076	95.14 / 0.2430
ResNet50	23,850,758	263,046	40.06	1.4701	37.96	1.5088	38.88 / 1.4945
EfficientNetB0	4,214,313	164,742	21.48	1.7750	21.51	1.7749	22.28 / 1.7758
DenseNet121	7,169,478	131,974	98.19	0.1904	99.17	0.1668	98.18 / 0.1931

and EfficientNetB0 is inherently unsuitable for this task, but that conclusion is not supported by this notebook alone. The shared $[0, 1]$ input scaling is not equivalent to each architecture’s recommended application preprocessing. The table therefore reports empirical behavior of the implemented pipeline, not an architecture-level universal ranking.

C. Separate VGG16 Baseline

The later VGG16 baseline experiment uses a different head (flattening followed by a 256-unit dense layer) and records 99.89% training accuracy, 99.72% validation accuracy, and 98.63% test accuracy, with corresponding losses of 0.0037,

0.0061, and 0.0594. This is the highest completed Keras test accuracy recorded in the notebook. It is deliberately not inserted as a fifth row in Table I, because its head and experimental context differ from the common-head benchmark.

The corresponding attention model never reaches training. Its construction raises a KerasTensor compatibility error in the CBAM implementation; a subsequent comparison cell then raises a NameError because `attention_model` was never created. Hence, there is no valid numerical baseline-versus-attention result.

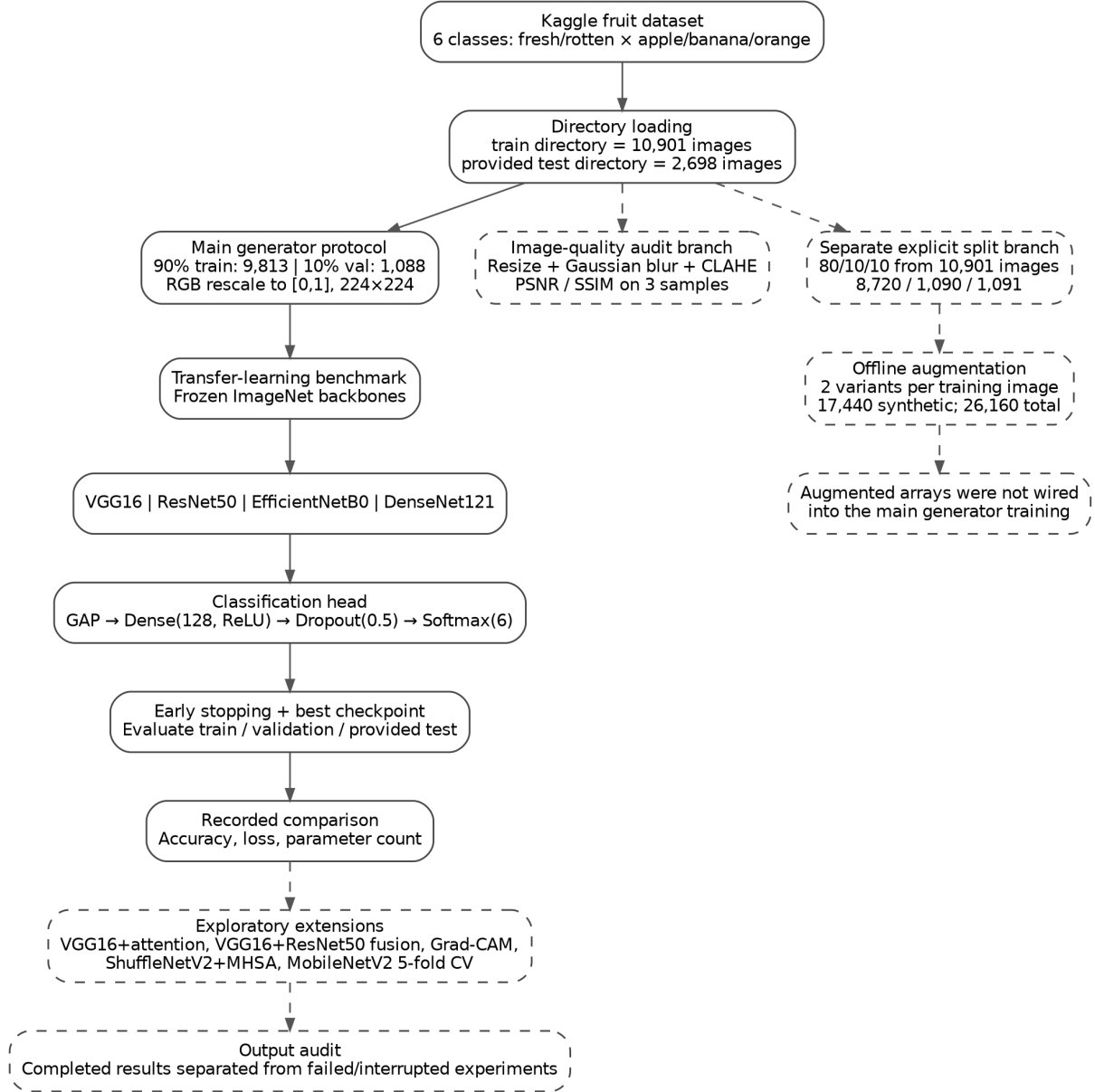


Fig. 2: Output-audited project methodology. Solid arrows denote the main generator-based benchmark; dashed branches denote separate image-quality, augmentation, and exploratory experiments.

D. Partial ShuffleNetV2–MHSA Training

The PyTorch ShuffleNetV2–MHSA branch records five completed epochs before interruption. Validation accuracy rises from 98.35% in epoch 1 to 99.50% in epochs 3–4, while training accuracy reaches 98.51% by epoch 5. The trajectory is shown in Fig. 5. Because the requested 50-epoch run and final test evaluation are absent, these values are treated as a partial learning curve rather than final model performance.

E. Output Validity Audit

Several later notebook outputs were excluded from the headline results. Table II summarizes why. This audit is

necessary because a large notebook can preserve outputs from cells whose underlying model object or data order has changed.

The final row deserves emphasis. The Keras test generator is created without `shuffle=False`. Later, `y_true=test_data.classes` is compared with predictions returned by `model.predict(test_data)`. If the generator traverses samples in shuffled order, the prediction and ground-truth arrays are not aligned. The resulting mean PPV, NPV, specificity, McNemar, and bootstrap accuracy outputs therefore cannot be treated as valid classifier performance measures. Their inconsistency with the preceding 98.18% DenseNet121 test accuracy further reinforces the need to

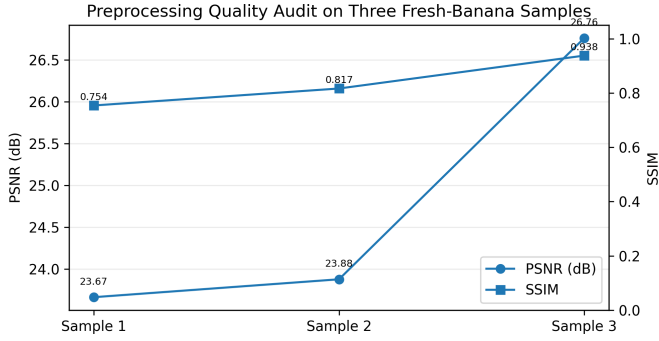


Fig. 3: Recorded PSNR and SSIM values for three preprocessing-audit samples.

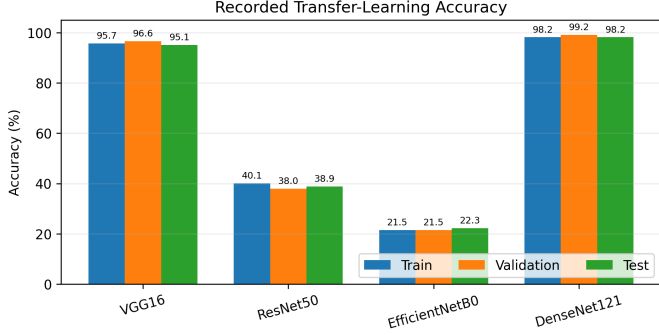


Fig. 4: Recorded train, validation, and test accuracies from the four completed main benchmark runs.

exclude them.

V. DISCUSSION

The most defensible finding is that the implemented DenseNet121 run is the strongest of the four common-head transfer-learning experiments, while the separate VGG16 baseline also performs strongly. DenseNet’s dense connectivity is designed to encourage feature reuse and gradient flow [15], which may be useful for visual texture differences between fresh and spoiled fruit. Nevertheless, the experiment does not isolate architecture as the only changing factor because the application-specific preprocessing expectations of the backbones are not standardized.

The project also shows why reproducibility involves more than saving a high accuracy value. The same notebook contains a successful augmentation routine that is disconnected from the main training pipeline, an attention branch that fails before model creation, a Grad-CAM branch that runs after a model-variable reassignment, and a statistical evaluation whose label/prediction alignment is questionable. Treating those cells as a single uninterrupted experimental narrative would overstate what was actually demonstrated.

These observations are consistent with the broader literature, where strong freshness-recognition results have been reported using transfer learning, tuned CNNs, multi-task models, and model fusion [3], [4], [6], [8]. However, direct accuracy comparison across papers is inappropriate without accounting for dataset composition, split strategy, class balance, augmentation, preprocessing, and evaluation protocol.

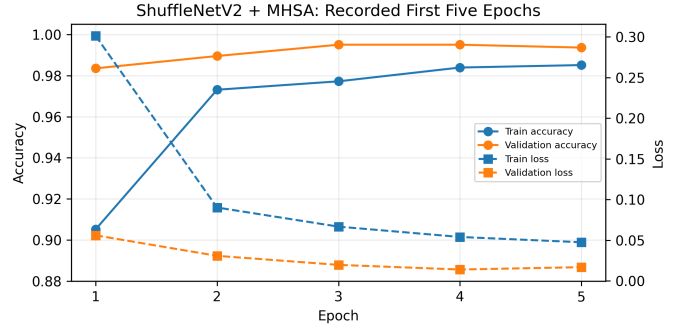


Fig. 5: Recorded first five epochs of the interrupted ShuffleNetV2-MHSA experiment.

TABLE II: Status of extended experiments in the recorded notebook

Branch	Recorded status and reporting decision
Offline augmentation	Completed; not wired into main generator training.
VGG16 + attention	Construction fails with KerasTensor/TensorFlow operation error; no result claimed.
VGG16+ResNet50 fusion	Model is constructed and predicts a batch; no complete final evaluation table.
Grad-CAM	Fails after <code>model</code> refers to a PyTorch object without Keras layers.
ShuffleNetV2-MHSA	Five epochs recorded; run interrupted before final test evaluation.
5-fold MobileNetV2 CV	Code present; no recorded fold results.
PPV/NPV/McNemar/bootstrap	Excluded from conclusions because test generator prediction order was not fixed while labels came from <code>test_data.classes</code> .

VI. LIMITATIONS AND REPRODUCIBILITY

The current study has several limitations. First, the main test generator uses the provided test directory but does not disable shuffling for post-hoc per-sample metrics. Future evaluation should instantiate the final test generator with `shuffle=False` and verify filename-label alignment before computing confusion matrices, class-specific metrics, or bootstrap statistics. Second, model-specific input preprocessing should be standardized according to each pretrained backbone before interpreting architectural differences. Third, the separately generated augmented dataset should either be connected explicitly to training or removed from claims about model performance. Fourth, the successful VGG16 baseline should be rerun under the exact same head, optimizer, callbacks, and preprocessing as the four-model benchmark if it is to be included in a strict leaderboard.

The exploratory branches also require completion. The CBAM implementation can be rebuilt with Keras layers or backend-safe operations; Grad-CAM should receive an explicit Keras model variable rather than a reused generic `model`; the ShuffleNetV2-MHSA run should complete all scheduled epochs and a held-out test evaluation; and the MobileNetV2 five-fold cross-validation code should be executed and its fold-level distribution reported. Cross-validation is particularly valuable when the goal is to estimate variability rather than a single split score.

VII. CONCLUSION

This paper reconstructs and audits a six-class fruit freshness classification notebook rather than presenting every cell as

a successful experiment. Under the main shared generator protocol, DenseNet121 recorded the best four-model test accuracy at 98.18%, while VGG16 achieved 95.14%. A separate VGG16 baseline with a different head reached 98.63% and is reported separately. The preprocessing audit recorded PSNR values of 23.67–26.76 dB and SSIM values of 0.754–0.938. The notebook further demonstrates useful exploratory directions—attention, feature fusion, Grad-CAM, lightweight attention, and cross-validation—but several of these remain incomplete or invalidated by runtime or evaluation-order issues. The central lesson is methodological: strong image-classification results are most useful when the data path, preprocessing, model state, and evaluation order are explicit enough that each reported number can be traced to a valid experimental run.

REFERENCES

- [1] S. S. S. Palakodati, V. R. Chirra, Y. Dasari, and S. Bulla, “Fresh and Rotten Fruits Classification Using CNN and Transfer Learning,” *Revue d’Intelligence Artificielle*, vol. 34, no. 5, pp. 617–622, 2020, doi: 10.18280/ria.340512.
- [2] L. G. Fahad, S. F. Tahir, U. Rasheed, H. Saqib, M. Hassan, and H. Alquhayz, “Fruits and Vegetables Freshness Categorization Using Deep Learning,” *Computers, Materials & Continua*, vol. 71, no. 3, pp. 5083–5098, 2022, doi: 10.32604/cmc.2022.023357.
- [3] U. Amin, M. I. Shahzad, A. Shahzad, M. Shahzad, U. Khan, and Z. Mahmood, “Automatic Fruits Freshness Classification Using CNN and Transfer Learning,” *Applied Sciences*, vol. 13, no. 14, Art. no. 8087, 2023, doi: 10.3390/app13148087.
- [4] F. Sia and N. S. Baco, “Hyperparameter Tuning of Convolutional Neural Network for Fresh and Rotten Fruit Recognition,” in *Proc. IEEE Int. Conf. Artificial Intelligence in Engineering and Technology (IICAET)*, 2023, pp. 112–116, doi: 10.1109/IICAET59451.2023.10291915.
- [5] R. Ahmed, M. M. Haque, S. Saha, C. Saha, M. Dutta, D. Z. Karim, M. Mostakim, and A. A. Rasel, “Fresh or Stale: Leveraging Deep Learning to Detect Freshness of Fruits and Vegetables,” in *Proc. Int. Conf. Electrical, Computer, Communications and Mechatronics Engineering (ICECCME)*, 2023, pp. 1–6, doi: 10.1109/ICECCME57830.2023.10252295.
- [6] Y. Zhang, X. Yang, Y. Cheng, X. Wu, X. Sun, R. Hou, and H. Wang, “Fruit freshness detection based on multi-task convolutional neural network,” *Current Research in Food Science*, vol. 8, Art. no. 100733, 2024, doi: 10.1016/j.crf.2024.100733.
- [7] S. Sofana Reka, A. Bagelika, P. Venugopal, V. Ravi, and H. Devarajan, “Deep Learning-Based Classification of Rotten Fruits and Identification of Shelf Life,” *Computers, Materials & Continua*, vol. 78, no. 1, pp. 781–794, 2024, doi: 10.32604/cmc.2023.043369.
- [8] S. T. Makmud, A. N. Haryanto, and S. Y. Prasetyo, “Improving rotten fruit classification accuracy through fusion of multiple pretrained CNN models,” *Engineering and Applied Science Research*, vol. 51, no. 3, pp. 313–320, 2024, doi: 10.14456/easr.2024.30.
- [9] Minavathi, A. P. Koundinya, V. Suresh, S. Girish, and Venkatesh M. G., “Classification of Fruits using Convolutional Neural Network,” *International Journal of Computer Applications*, vol. 186, no. 22, pp. 37–41, 2024, doi: 10.5120/ijca2024923661.
- [10] D. Ghosh and J. P. Singh, “A hybrid CNN-LSTM model for accurate fruit freshness classification using deep learning,” *Discover Artificial Intelligence*, vol. 6, Art. no. 134, 2026, doi: 10.1007/s44163-025-00750-7.
- [11] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A Large-Scale Hierarchical Image Database,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2009, pp. 248–255, doi: 10.1109/CVPR.2009.5206848.
- [12] S. J. Pan and Q. Yang, “A Survey on Transfer Learning,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010, doi: 10.1109/TKDE.2009.191.
- [13] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in *Proc. International Conference on Learning Representations (ICLR)*, 2015.
- [14] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.
- [15] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, “Densely Connected Convolutional Networks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2017, pp. 2261–2269, doi: 10.1109/CVPR.2017.243.
- [16] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *Proc. 36th International Conference on Machine Learning*, vol. 97, 2019, pp. 6105–6114.
- [17] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520, doi: 10.1109/CVPR.2018.00474.
- [18] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, “ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design,” in *Proc. European Conference on Computer Vision*, 2018, pp. 116–131, doi: 10.1007/978-3-030-01264-9_8.
- [19] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, “CBAM: Convolutional Block Attention Module,” in *Proc. European Conference on Computer Vision*, 2018, pp. 3–19, doi: 10.1007/978-3-030-01234-2_1.
- [20] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual Explanations From Deep Networks via Gradient-Based Localization,” in *Proc. IEEE International Conference on Computer Vision*, 2017, pp. 618–626, doi: 10.1109/ICCV.2017.74.
- [21] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [22] C. Shorten and T. M. Khoshgoftaar, “A survey on Image Data Augmentation for Deep Learning,” *Journal of Big Data*, vol. 6, Art. no. 60, 2019, doi: 10.1186/s40537-019-0197-0.
- [23] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004, doi: 10.1109/TIP.2003.819861.
- [24] A. Hore and D. Ziou, “Image Quality Metrics: PSNR vs. SSIM,” in *Proc. 20th International Conference on Pattern Recognition*, 2010, pp. 2366–2369, doi: 10.1109/ICPR.2010.579.
- [25] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. International Conference on Learning Representations (ICLR)*, 2015.